

CSCI-B 651 NLP Spring 2026 Homework 1

January 16, 2026

Instructions

We will be using Canvas to collect your assignments. Please read the following instructions to prepare your submission.

1. Submit your solution in a pdf file and a zip file (<yourLastName_FirstName>.pdf/zip). Your write-up must be in pdf. Your code must be in the zip file.
2. In your pdf file, the solution to each problem should start on a new page.
3. Latex is strongly encouraged to write your solutions, e.g., using Overleaf (<https://www.overleaf.com/>). However, scanned handwritten copies are also acceptable. Hard copies will not be accepted.
4. You may discuss the problems and potential directions for solving them with another student. However, you need to write your own solutions and code separately, and not as a group activity. Please list the students you collaborated with on your submission.

Problem 1 (20 points)

Write regular expressions for the following cases.

- (a) One or more words (only with lowercase alphabets) separated by spaces
e.g., “red blue green white”
`/[a-z]+(\s[a-z]+)*`
- (b) Title case sentences, assuming all words are capitalized. Note that the text can contain numbers and punctuation.
e.g., “Why Sleep Is So Important To Your Health?”
`/[A-Z][A-Za-z0-9]*[?!,.]*(\s+[A-Z][A-Za-z0-9]*[?!,.]*)*/`
- (c) Strings that contain the word “ice” without matching the words that contain “ice” such as “icecream” or “ice-bucket”
`/\bice\b/`
- (d) The set of all lowercase alphabetic strings ending in a “b”
`/^[a-z]*b$/`
- (e) All strings that start at the beginning of the line with an integer and that end at the end of the line with a word
`/^[0-9]+.*[A-Za-z]+$`

Problem 2 (Programming involved, 28 points)

Download the Diplomacy training dataset (train.jsonl): <https://sites.google.com/view/qanta/projects/diplomacy>

We will only use the language data in the “messages” field in the JSON format. First, read the dataset (train.jsonl) and extract only the “messages” using a JSON parser. Write a new file (“data.txt”) which contains messages in each line. We will use **data.txt** from now on.

Use the NLTK package (<https://www.nltk.org/>) to split the data into sentences. Use the `sent_tokenize()` function.

- (a) How many sentences are there? Note that you need to ignore empty lines and empty sentences.
There’re 17901 sentences.

Now let’s find words. First, split the sentences using the python `split(' ’)` function.

- (b) How many tokens are there?
There’re 272336 tokens.

This time, use NLTK’s `word_tokenize()` function to split into words.

- (c) How many tokens are there now?
There’re 321214 tokens.

Lowercase all the words.

- (d) How many tokens and types now?
There’re 321214 tokens and 8780 types.
- (e) Compare the number of tokens from (b), (c), and (d). Why are they different?
(b) keeps punctuation attached to words, (c) separates punctuation, and (d) number of tokens stays the same as c, but types decrease, because ‘The’ and ‘the’ are now counted as the same type.

Lastly, make a dictionary of word type counts. The dictionary contains word type as its key, and frequency as its value. Sort the dictionary.

- (f) What is the most frequent word type?
It’s word ‘i’.
- (g) What is the 5th most frequent word type?
It’s word ‘you’.
- (h) Using the sorted dictionary, draw a graph to see if this data shows the Zipf’s law. X-axis: ranked words, Y-axis: frequencies. Submit your graph and discuss it (in your pdf). Submit your code as well.
The log-log plot displays a straight line with negative slope, confirming that word frequency follows Zipf’s Law: $f(r) \propto \frac{1}{r}$, where $f(r)$ is frequency and r is rank.

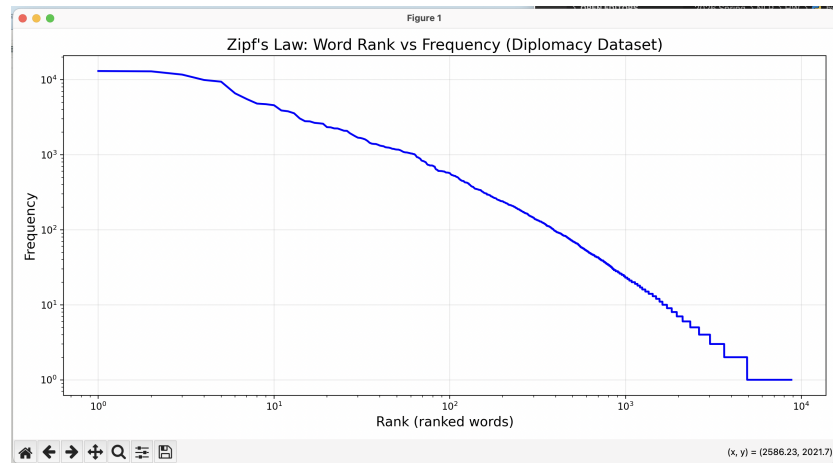
Observations:

- **Few high-frequency words:** A small number of function words (“the”, “to”, “a”) dominate the corpus, appearing thousands of times.
- **Long tail of rare words:** Most words appear only a few times, creating a long tail distribution.

- **Power-law relationship:** The linear relationship on log-log scale indicates a power-law distribution, characteristic of natural language.

Implications:

- Common words can be compressed efficiently with shorter codes
- Stop words (high-frequency function words) often need filtering in NLP tasks
- Rare words continue appearing even in large corpora, making complete vocabulary coverage difficult
- This pattern is universal across languages, reflecting fundamental properties of human communication



Problem 3 (24 points)

We are given the following corpus:

```
<s> I am Tom </s>
<s> Tom I am </s>
<s> I am Tom </s>
<s> I am Tom and she is Jane </s>
<s> I like pink eggs and Tom </s>
<s> I like Tom </s>
<s> Tom like Jane </s>
```

- (a) Using a bigram language model without smoothing, what is $P(\text{Tom}|\text{am})$? Include $\langle s \rangle$ and $\langle /s \rangle$ in your counts just like any other token. Please show your work – how you get your answer.

$$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

$$P(\text{Tom} | \text{am}) = \frac{\text{Count}(\text{am Tom})}{\text{Count}(\text{am})} = \frac{3}{4}$$

- (b) Using a bigram language model with add-one smoothing, what is $P(\text{Tom}|\text{am})$? Include $\langle s \rangle$ and $\langle /s \rangle$ in your counts just like any other token. Please show your work – how you get your answer.

$$V = 12$$

$$P_{\text{add-1}}(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i) + 1}{\text{Count}(w_{i-1}) + V}$$

$$P_{\text{add-1}}(\text{Tom} | \text{am}) = \frac{\text{Count}(\text{am Tom}) + 1}{\text{Count}(\text{am}) + V}$$

$$= \frac{3 + 1}{4 + 12}$$

$$= \frac{1}{4}$$

- (c) If we use linear interpolation smoothing between a maximum-likelihood bigram model and a maximum-likelihood unigram model with $\lambda_1 = 1/2$ and $\lambda_2 = 1/2$, what is $P(\text{Tom}|\text{am})$? Include $\langle s \rangle$ and $\langle /s \rangle$ in your counts just like any other token. Please show work – how you get your answer.

$$P_{\text{interpolation}}(w_i | w_{i-1}) = \lambda_1 P(w_i) + \lambda_2 P(w_i | w_{i-1})$$

$$P_{\text{interpolation}}(\text{Tom} | \text{am}) = \lambda_1 P(\text{Tom}) + \lambda_2 P(\text{Tom} | \text{am})$$

$$= \frac{1}{2} \left(\frac{\text{Count}(\text{Tom})}{N} \right) + \frac{1}{2} \left(\frac{\text{Count}(\text{am Tom})}{\text{Count}(\text{am})} \right)$$

$$= \frac{1}{2} \left(\frac{7}{42} \right) + \frac{1}{2} \left(\frac{3}{4} \right)$$

$$= \frac{11}{24}$$

- (d) Using the above model from (a), what will be the probability of “ $\langle s \rangle$ I like Jane $\langle /s \rangle$ ”? Please

show work – how you get your answer.

$$\begin{aligned}
P(< s > | I \text{ like Jane } < /s >) &= P(I | < s >) \times P(\text{like} | I) \times P(\text{Jane} | \text{like}) \times P(< /s > | \text{Jane}) \\
&= \frac{\text{Count}(< s > | I)}{\text{Count}(< s >)} \times \frac{\text{Count}(I \text{ like})}{\text{Count}(I)} \times \frac{\text{Count}(\text{like Jane})}{\text{Count}(\text{like})} \times \frac{\text{Count}(\text{Jane } < /s >)}{\text{Count}(\text{Jane})} \\
&= \frac{5}{7} \times \frac{2}{6} \times \frac{1}{3} \times \frac{2}{2} \\
&= \frac{5}{63}
\end{aligned}$$

- (e) Using the above model from (b), what will be the probability of “<s> I like Jane </s>”? Please show work – how you get your answer.

$$\begin{aligned}
P_{add-1}(< s > | I \text{ like Jane } < /s >) &= P_{add-1}(I | < s >) \times P_{add-1}(\text{like} | I) \times P_{add-1}(\text{Jane} | \text{like}) \times P_{add-1}(< /s > | \text{Jane}) \\
&= \frac{\text{Count}(< s > | I) + 1}{\text{Count}(< s >) + V} + \frac{\text{Count}(I \text{ like}) + 1}{\text{Count}(I) + V} + \frac{\text{Count}(\text{like Jane}) + 1}{\text{Count}(\text{like}) + V} + \frac{\text{Count}(\text{Jane } < /s >) + 1}{\text{Count}(\text{Jane}) + V} \\
&= \frac{5 + 1}{7 + 12} \times \frac{2 + 1}{6 + 12} \times \frac{1 + 1}{3 + 12} \times \frac{2 + 1}{2 + 12} \\
&= \frac{6}{19} \times \frac{3}{18} \times \frac{2}{15} \times \frac{3}{14} \\
&= \frac{1}{665}
\end{aligned}$$

- (f) Using the above model from (c), what will be the probability of “<s> I like Jane </s>”? Please

show work – how you get your answer.

$$P_{inter}(< s > \text{ I like Jane } < /s >) = P_{inter}(\text{I} \mid < s >) \times P_{inter}(\text{like} \mid \text{I}) \times P_{inter}(\text{Jane} \mid \text{like}) \times P_{inter}(< /s > \mid \text{Jane})$$

$$P_{inter}(\text{I} \mid < s >) = \lambda_1 P(I) + \lambda_2 P(\text{I} \mid < s >)$$

$$= \frac{1}{2} \left(\frac{\text{Count}(I)}{N} \right) + \frac{1}{2} \left(\frac{\text{Count}(< s > I)}{\text{Count}(< s >)} \right)$$

$$= \frac{1}{2} \left(\frac{6}{42} \right) + \frac{1}{2} \left(\frac{5}{7} \right)$$

$$= \frac{6}{14}$$

$$P_{inter}(\text{like} \mid \text{I}) = \lambda_1 P(\text{like}) + \lambda_2 P(\text{like} \mid \text{I})$$

$$= \frac{1}{2} \left(\frac{\text{Count}(\text{like})}{N} \right) + \frac{1}{2} \left(\frac{\text{Count}(\text{I like})}{\text{Count}(\text{I})} \right)$$

$$= \frac{1}{2} \left(\frac{3}{42} \right) + \frac{1}{2} \left(\frac{2}{6} \right)$$

$$= \frac{17}{84}$$

$$P_{inter}(\text{Jane} \mid \text{like}) = \lambda_1 P(\text{Jane}) + \lambda_2 P(\text{Jane} \mid \text{like})$$

$$= \frac{1}{2} \left(\frac{\text{Count}(\text{Jane})}{N} \right) + \frac{1}{2} \left(\frac{\text{Count}(\text{like Jane})}{\text{Count}(\text{like})} \right)$$

$$= \frac{1}{2} \left(\frac{2}{42} \right) + \frac{1}{2} \left(\frac{1}{3} \right)$$

$$= \frac{8}{42}$$

$$P_{inter}(< /s > \mid \text{Jane}) = \lambda_1 P(< /s >) + \lambda_2 P(< /s > \mid \text{Jane})$$

$$= \frac{1}{2} \left(\frac{\text{Count}(< /s >)}{N} \right) + \frac{1}{2} \left(\frac{\text{Count}(\text{Jane } < /s >)}{\text{Count}(\text{Jane})} \right)$$

$$= \frac{1}{2} \left(\frac{7}{42} \right) + \frac{1}{2} \left(\frac{2}{2} \right)$$

$$= \frac{7}{12}$$

$$P_{inter}(< s > \text{ I like Jane } < /s >) = \frac{6}{14} \times \frac{17}{84} \times \frac{8}{42} \times \frac{7}{12} = \frac{17}{1764}$$

Problem 4 (Programming involved, 28 points)

For this problem, we will use Python 3.

First, we are going to download the Brown corpus using NLTK.

```
from nltk.corpus import brown
```

```
news_data = brown.sents(categories='news')
romance_data = brown.sents(categories='romance')
```

Note that the texts are already split into sentences and also are tokenized.

Write a program to compute unsmoothed unigram and bigram models.

Before doing that, you need to lowercase all the words.

In addition, remove tokens that only consist of punctuation. For example, change ['Phil', 'nodded', '.'] into ['Phil', 'nodded'] before adding <s> and </s>. Therefore, ['Phil', 'nodded', '.'] will become ['<s>', 'Phil', 'nodded', '</s>']

Don't remove tokens that consist of both alphanumeric characters and punctuation. e.g., Richard's, 30\$, Inc.

Finally, you need to include <s> and </s> before and after each sentence.

You can use the `nltk.util` package, but don't use the `nltk.lm` package, which means you need write your own function to create vocabularies, calculate MLE, etc.

Run your program on the news data and the romance data. Now compare the statistics of the two corpora.

- (a) How many non-zero unigrams (in terms of counts) did you get for each corpus?
News corpus: 13101 unique unigrams
Romance corpus: 7872 unique unigrams
- (b) How many non-zero bigrams (in terms of counts) did you get for each corpus?
News corpus: 59856 unique bigrams
Romance corpus: 36384 unique bigrams
- (c) List the 10 most common unigrams (in terms of counts) from each dataset with their probabilities $P(w_t)$ (using MLE). You can create a table to show the numbers. Any interesting differences between the two?
 - 1. Function Words vs Content Words
News corpus: Dominated by articles, prepositions, and formal markers (e.g., "the", "of", "to", "in", "a").
Romance corpus: More pronouns, especially personal and possessive pronouns reflecting character interactions (e.g., "she", "he", "her", "his", "I").
 - 2. Modal Verbs
News: Higher frequency of "will" (future events, predictions, official statements).
Romance: Higher frequency of "could" and "would" (hypothetical situations, emotions, desires).
 - 3. Sentence Markers
Both will have <s> and </s> in top 10, but their relative frequencies differ based on average sentence length.
News: Longer sentences, so sentence markers have relatively lower frequency.
Romance: More dialogue and shorter sentences, so sentence markers appear more frequently.
 - 4. Lexical Density

News: Higher noun ratio, more information-dense with formal vocabulary.

Romance: More varied emotional and descriptive vocabulary, but top unigrams still dominated by common function words.

News Corpus		
corpus	counts	probability
the	6386	0.065271
<s>	4623	0.047252
</s>	4623	0.047252
of	2861	0.029242
and	2186	0.022343
to	2144	0.021914
a	2130	0.021771
in	2020	0.020646
for	969	0.009904
that	829	0.008473

Romance Corpus		
corpus	counts	probability
<s>	4431	0.065659
</s>	4431	0.065659
the	2988	0.044277
and	1905	0.028228
to	1517	0.022479
a	1383	0.020493
of	1202	0.017811
he	1068	0.015826
was	999	0.014803
i	951	0.014092

- (d) List the 10 most common bigrams (in terms of counts) from each dataset with their probabilities $P(w_t|w_{t-1})$ (using MLE). You can create a table to show the numbers. Any interesting differences between the two?

1. Sentence Boundaries

News corpus: High frequency of (<s>, "the"), (<s>, "a"), (<s>, "in") - formal sentence openings with articles and prepositions.

Romance corpus: High frequency of (< s >, "she"), (<s>, "he"), (<s>, "i") - sentences often start with character pronouns, reflecting narrative focus on people.

2. Prepositional Phrases

News: More bigrams like ("of", "the"), ("in", "the"), ("to", "the") - formal, information-dense structures typical of news reporting.

Romance: Fewer prepositional phrase bigrams in top 10, replaced by character-action combinations.

3. Character Interactions

News: Minimal personal pronoun bigrams; focus on institutions and events.

Romance: Bigrams like ("she", "was"), ("he", "was"), ("her", "eyes"), ("his", "hand") - describing characters and their actions/attributes.

4. Verb Patterns

News: Bigrams with "will be", "has been", "have been" - reporting completed or future events with formal verb structures.

Romance: Bigrams with "could see", "would have", "had been" - narrative storytelling with past tense and emotional descriptions.

5. Named Entities vs Pronouns

News: More bigrams involving proper nouns and titles (e.g., locations, organization names).

Romance: Dominated by pronoun-based bigrams due to continuous character reference throughout dialogue and narrative.

News Corpus		
bigram	counts	probability
('of', 'the')	849	0.296749
('<s>', 'the')	780	0.168722
('in', 'the')	589	0.291584
('to', 'the')	277	0.129198
('on', 'the')	253	0.366136
('for', 'the')	220	0.227058
('at', 'the')	196	0.308176
('<s>', 'he')	192	0.041531
('will', 'be')	157	0.403599
('that', 'the')	148	0.178528

Romance Corpus		
bigram	counts	probability
('<s>', 'i')	386	0.087114
('<s>', 'he')	372	0.083954
('in', 'the')	273	0.293548
('<s>', 'she')	244	0.055067
('of', 'the')	235	0.195507
('<s>', 'the')	230	0.051907
('it', 'was')	179	0.249651
('<s>', 'it')	154	0.034755
('<s>', 'but')	144	0.032498
('he', 'was')	142	0.132959

Write a function to compute a probability of a given sentence using a n-gram model you built above.

- (e) What is the probability for “<s> I loved her when she laughed </s>” when using the bigram model from the news data?
The probability is 0.
- (f) What is the probability for “<s> I loved her when she laughed </s>” when using the bigram model from the romance data?
 $\log P = -27.412393762200526$

Add an option to your program to do add-one smoothing.

- (g) After applying add-one smoothing to your bigram models, what are the probabilities for “<s> I loved her when she laughed </s>” when using the model from the news data and the model from the romance data, respectively?
News model: $\log P = -61.14609797533516$
Romance model: $\log P = -49.47541030409295$